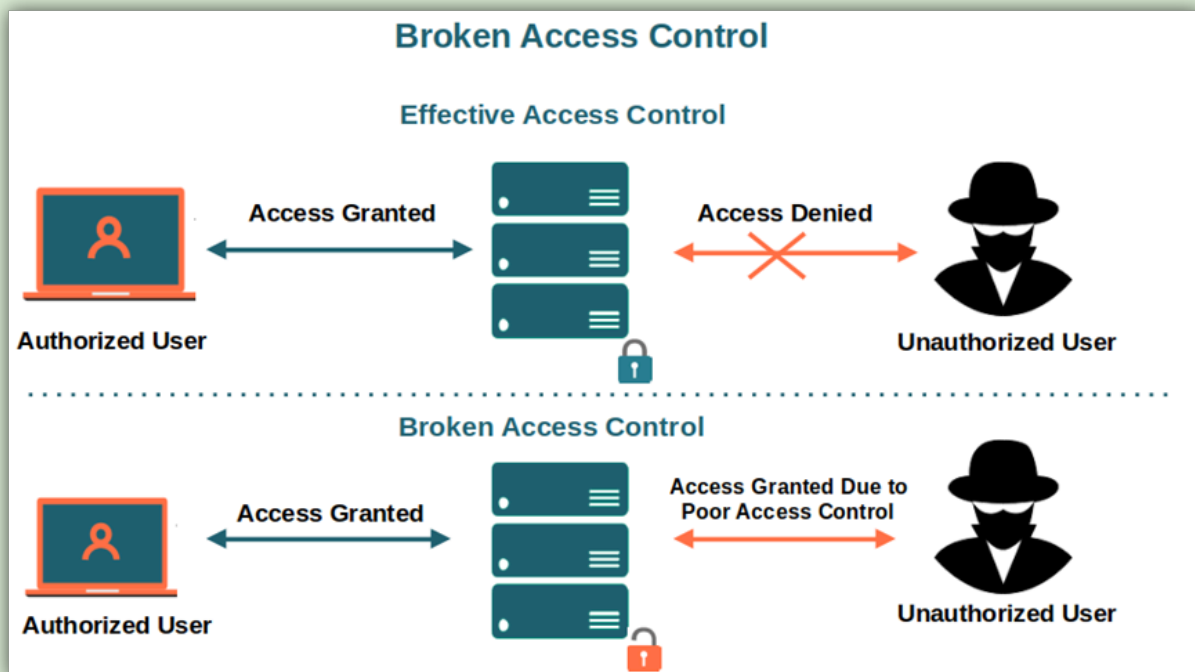


Authorization Bypass (Broken Access Control)

Authorization (authz) determines what an authenticated user can do. A bypass occurs when the application fails to enforce proper checks, letting users act outside their intended permissions.

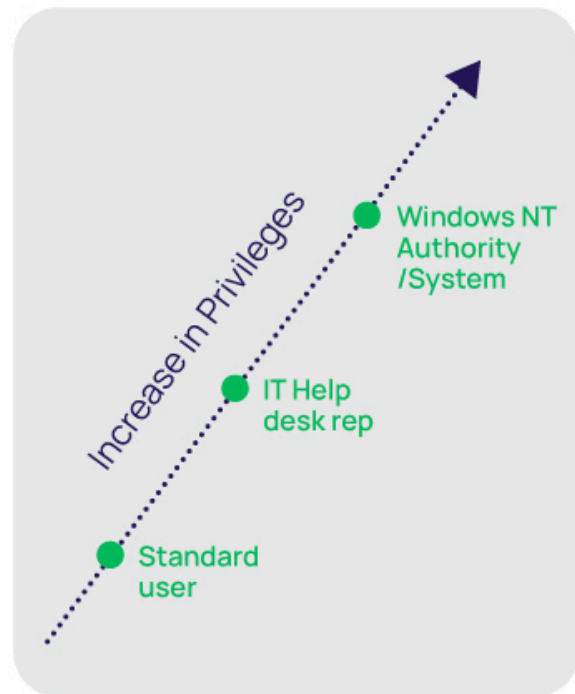


Common forms include:

- **Insecure Direct Object Reference (IDOR):** Direct manipulation of identifiers (e.g., `user_id=123` → `124`).
- **Vertical Privilege Escalation:** Low-priv user accesses admin functions.
- **Horizontal Privilege Escalation:** Same-priv level access to another user's data.
- **Forced Browsing:** Accessing hidden endpoints without checks.
- **Missing Function-Level Access Control:** No server-side verification.



Horizontal Privilege Escalation Attack

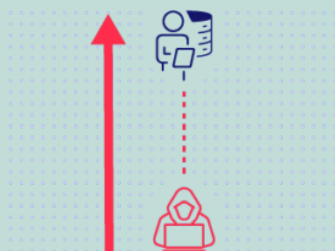


Vertical Privilege Escalation Attack

Types of Privilege Escalation

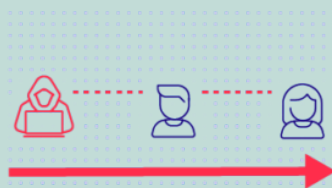
cymulate

Vertical Privilege Escalation



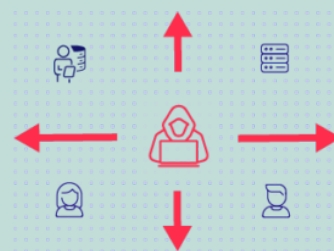
Attackers elevate from **standard** user to **admin or root** by exploiting vulnerabilities, gaining full control to modify settings and access sensitive data.

Horizontal Privilege Escalation



Attackers stay at the **same privilege level** but access other users' data or resources due to weak access controls or session management flaws.

Hybrid Privilege Escalation

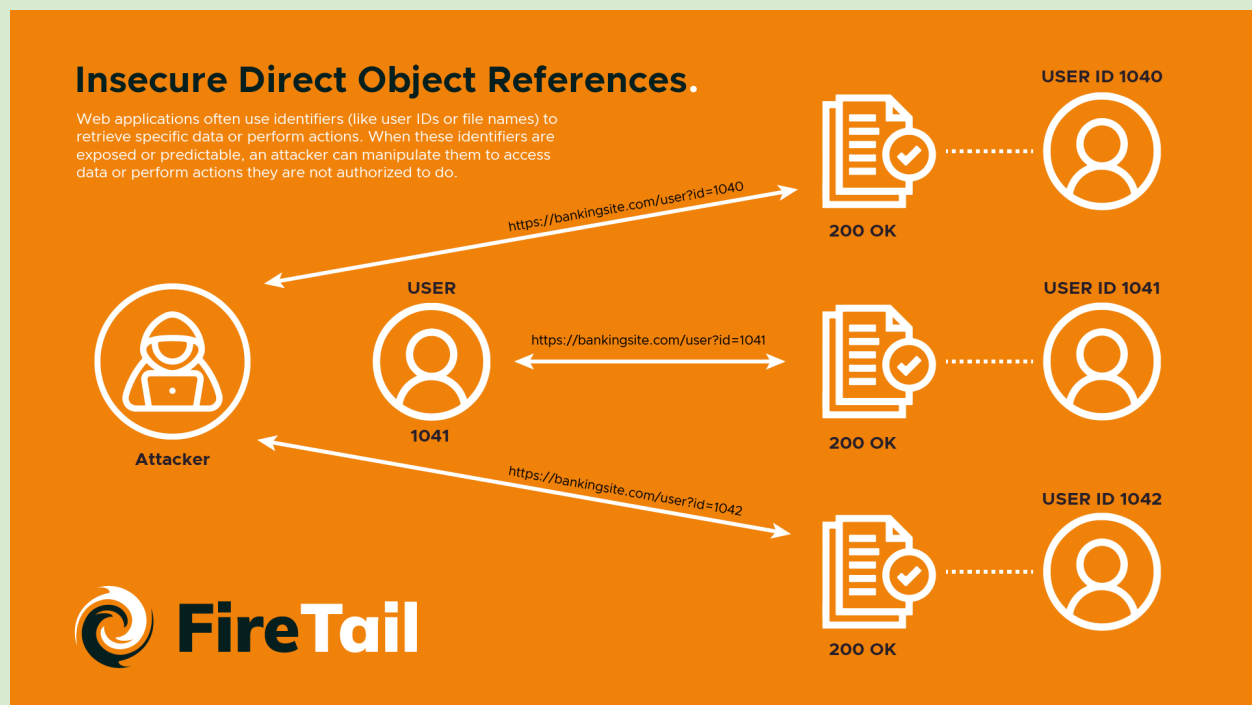


In complex environments like hybrid clouds, attackers exploit misconfigurations and weak IAM policies to escalate **privileges across on-premises and cloud systems**.

History & Evolution

- **Past:** Known as "Broken Access Control" in earlier OWASP lists, often #5. IDOR was a separate category.

- **Present:** Jumped to #1 in OWASP 2021 and held in 2025 release. 100% of tested apps showed some form. Recent examples (2025–early 2026):
 - CVE-2025-13526: IDOR in OneClick Chat to Order WordPress plugin – exposed customer data.
 - CVE-2025-63065: Authorization bypass in Media Library Assistant plugin.
 - CVE-2026-24544: Broken access in HD Quiz allowing unauthorized actions.
 - CVE-2026-1251: IDOR in SupportCandy ticket attachments.
 - Multiple WordPress and CMS plugins affected.
- **Future Trends:** Rising with APIs, microservices, and GraphQL. Automated tools and policy-as-code (e.g., OPA, Cerbos) will help, but complexity keeps it #1.



IDOR Attacks and the Growing Threat to Your API Security - FireTail blog posts

How It Works (Step-by-Step)

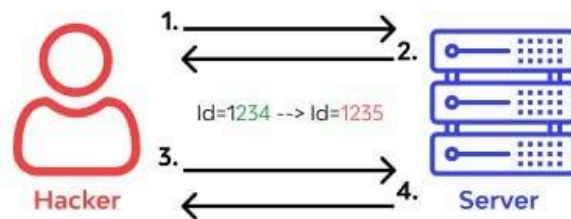
1. User authenticates → receives session/role.
2. Requests resource/action (e.g., GET /api/user/123).
3. Vulnerable server skips or weakly checks ownership/role.
4. Server grants access → bypass successful.

Insecure Direct Object Reference (IDOR) Vulnerability

1. Hacker identifies web application using direct object reference(s) and requests verified information

2. Valid http request is executed and direct object reference entity is revealed

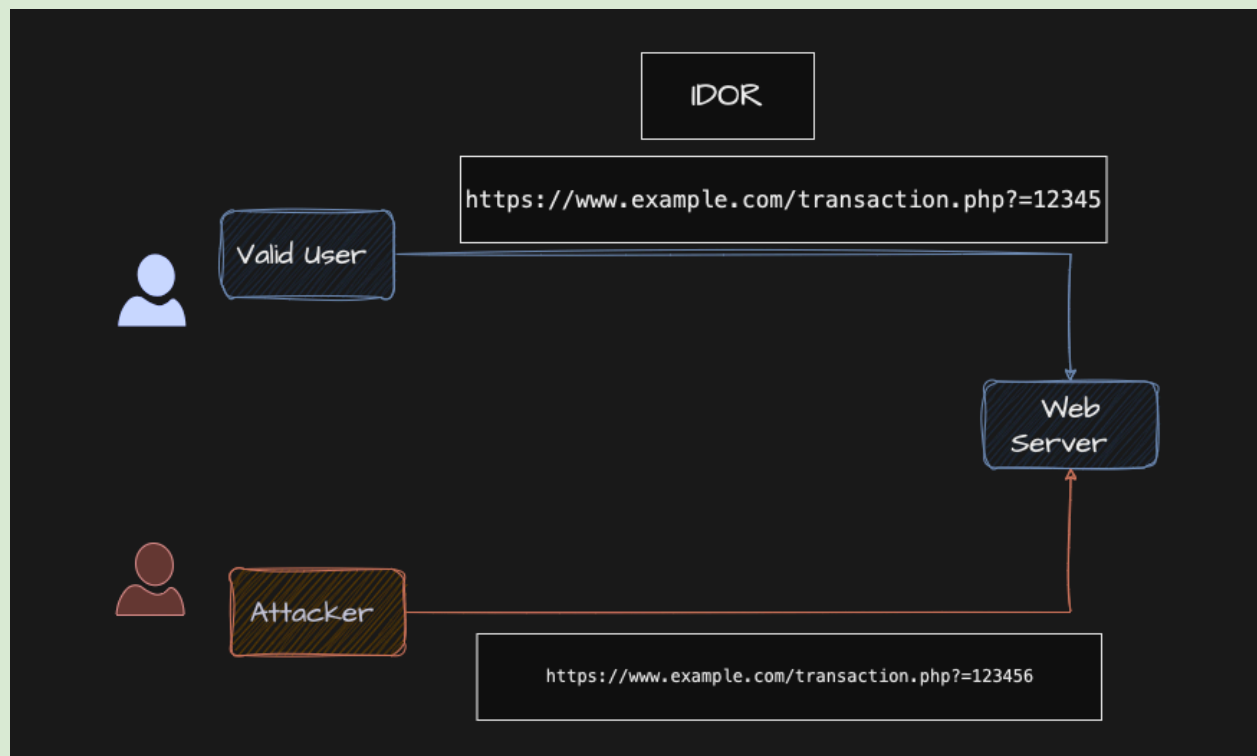
`https://example.com/account?id=1234`



`https://example.com/account?id=1235`

3. Direct object reference entity is manipulated and http request is performed again

4. http request is performed without user verification and hacker is granted access to sensitive information



Exploitation Procedure with Practical Example

Classic IDOR Example:

Vulnerable endpoint: `https://site.com/profile?user_id=1001` (shows your profile).

Attack:

1. Log in as low-priv user (your ID: 1001).
2. Intercept with Burp.
3. Change to `user_id=1002` (another user) or `user_id=1` (admin).
4. If no server-side check → view/edit others' data.

Vertical Escalation:

- Change POST parameter `role=user` to `role=admin`.
- Access `/admin/dashboard` directly.

Mass Assignment (related): Bind user input to objects without allowlisting → escalate by setting `"is_admin": true`.

Differences from Similar Vulnerabilities

Vulnerability	Focus	Requires Auth?	Typical Vector	Primary Impact
Authorization Bypass	What you can do	Usually yes	Parameter/URL manipulation	Data access/escalation
Broken Authentication	Who you are	No	Weak passwords/sessions	Impersonation
XSS/CSRF	Client-side tricks	Varies	Injected code/forged req	Session hijack/actions
Server-Side Injection	Code/DB execution	No	Malicious input	RCE/data leak

Authz bypass is server-side policy failure.

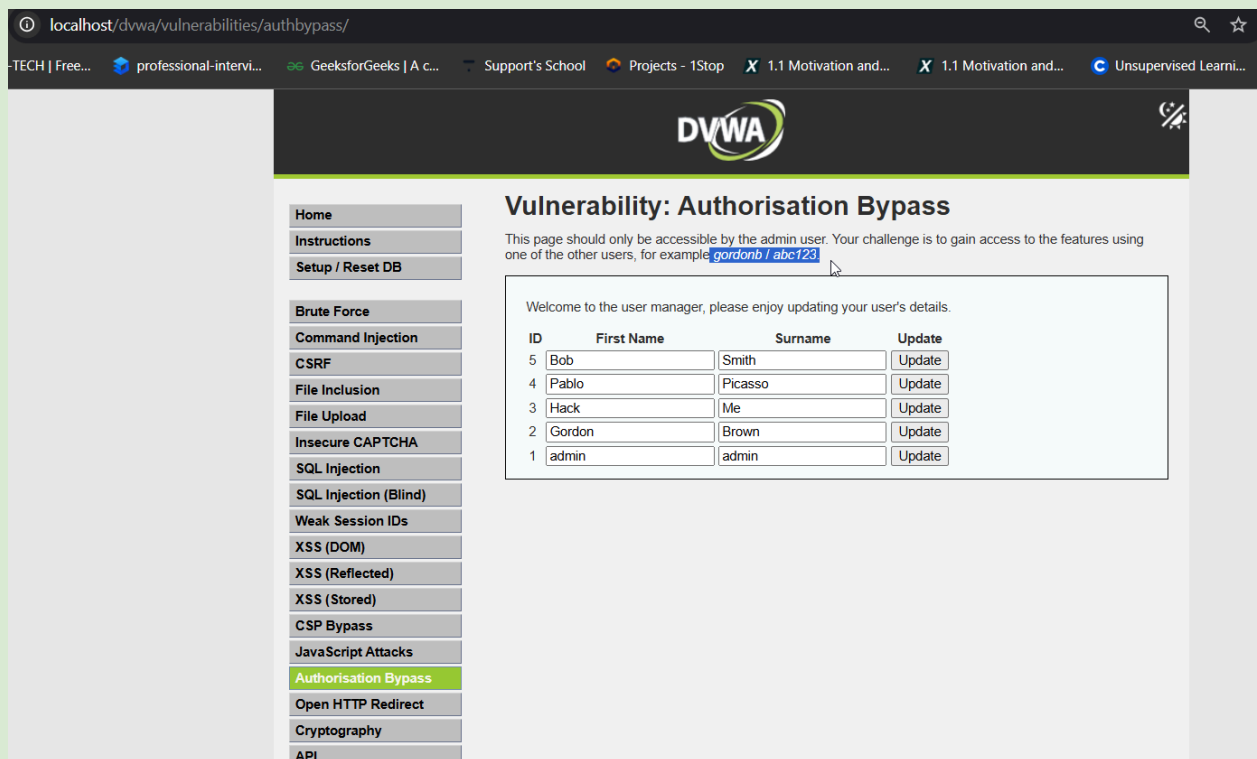
Impacts & Real-World Effects

- Sensitive data exposure (PII, financial).
- Account takeover/modification.
- Full system compromise (vertical to admin).
- Compliance violations (GDPR, etc.).
- In 2025–2026 plugin cases: Customer data leaks, unauthorized ticket access.

Tools:

- **Burp Suite:** Authorize extension (tests as different roles), Repeater for param manipulation.
- **OWASP ZAP:** Active scanner + manual exploration.
- **Automated:** Nuclei templates for IDOR/access control.
- **API Testing:** Postman + authorization matrices.

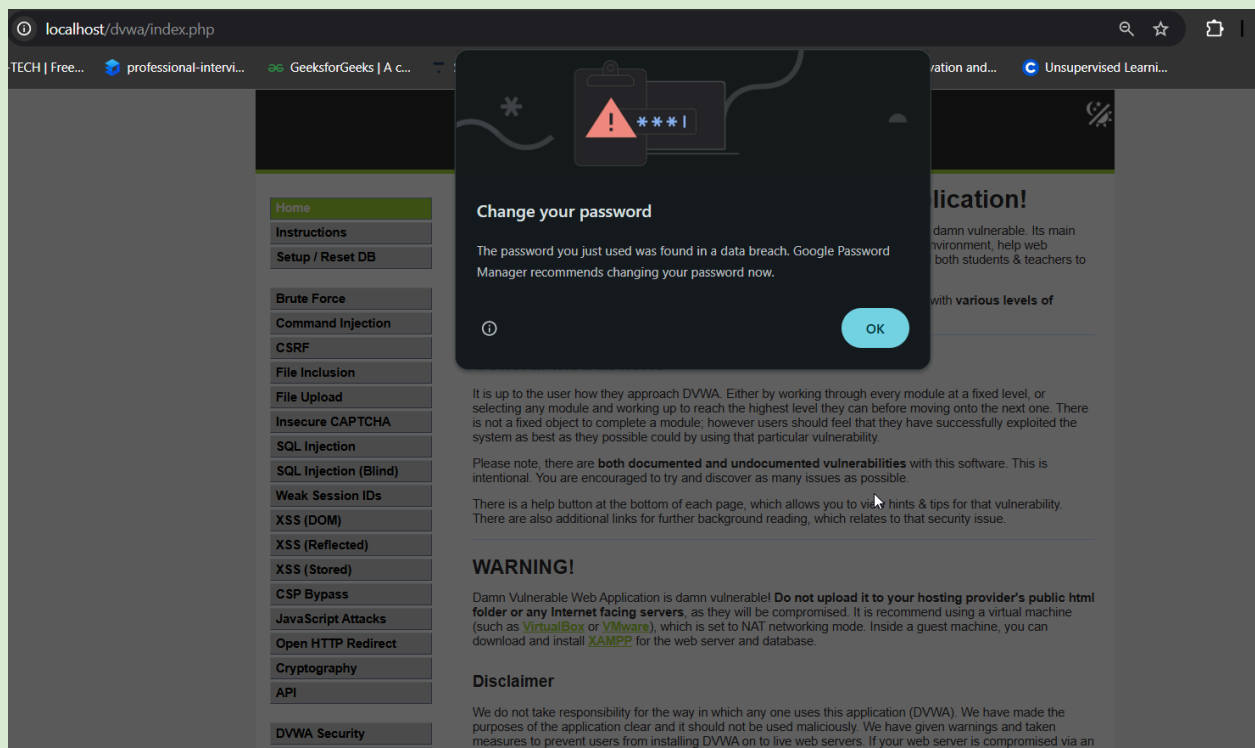
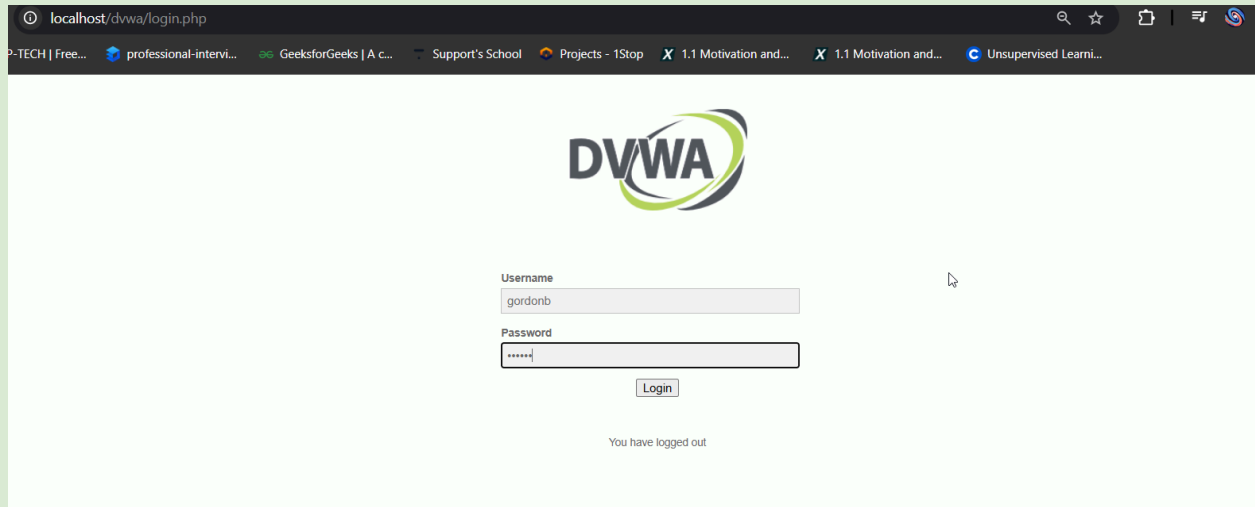
Exercise:



The screenshot shows the DVWA web application running on localhost. The browser address bar is `localhost/dvwa/vulnerabilities/authbypass/`. The page title is "Vulnerability: Authorisation Bypass". Below the title, a message states: "This page should only be accessible by the admin user. Your challenge is to gain access to the features using one of the other users, for example `gordonb / abc123`".

On the left sidebar, the "Authourisation Bypass" menu item is highlighted. The main content area contains a welcome message and a table of users:

ID	First Name	Surname	Update
5	Bob	Smith	Update
4	Pablo	Picasso	Update
3	Hack	Me	Update
2	Gordon	Brown	Update
1	admin	admin	Update



localhost/dvwa/

TECH | Free... professional-intervi... GeeksforGeeks | A c... Support's School Projects - 1Stop X 1.1 Motivation and... X 1.1 Motivation and... C Unsupervised Learn...

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with **various levels of difficulty**, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any Internet facing servers**, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

We do not take responsibility for the way in which any one uses this application (DVWA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DVWA on to live web servers. If your web server is compromised via an installation of DVWA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

DVWA aims to cover the most commonly seen vulnerabilities found in today's web applications. However there are plenty of other issues with web applications. Should you wish to explore any additional attack vectors, or want more difficult challenges, you may wish to look into the following other projects:

← → ↺ ⓘ localhost/dvwa/vulnerabilities/authorisation_bypass

Open P-TECH | Free... professional-intervi... GeeksforGeeks | A c... Support's School Projects - 1Stop

Not Found

The requested URL was not found on this server.

Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12 Server at localhost Port 80

Summary:

Authorization Bypass (Broken Access Control) tops OWASP in 2026 due to app complexity and developer oversights. Recent 2025–2026 CVEs in plugins highlight its prevalence. Master server-side checks, indirect references, and tools like Burp Authorize.

Key takeaway: **Authentication proves who you are—authorization must prove what you're allowed to do. Enforce it server-side, always.**